

bok25

**基
础**

班、计算机组成原理



寻址方式

4-2 寻址方式
@Beokayy_

1.指令的寻址方式

2.数据的寻址方式

Presented with xmind



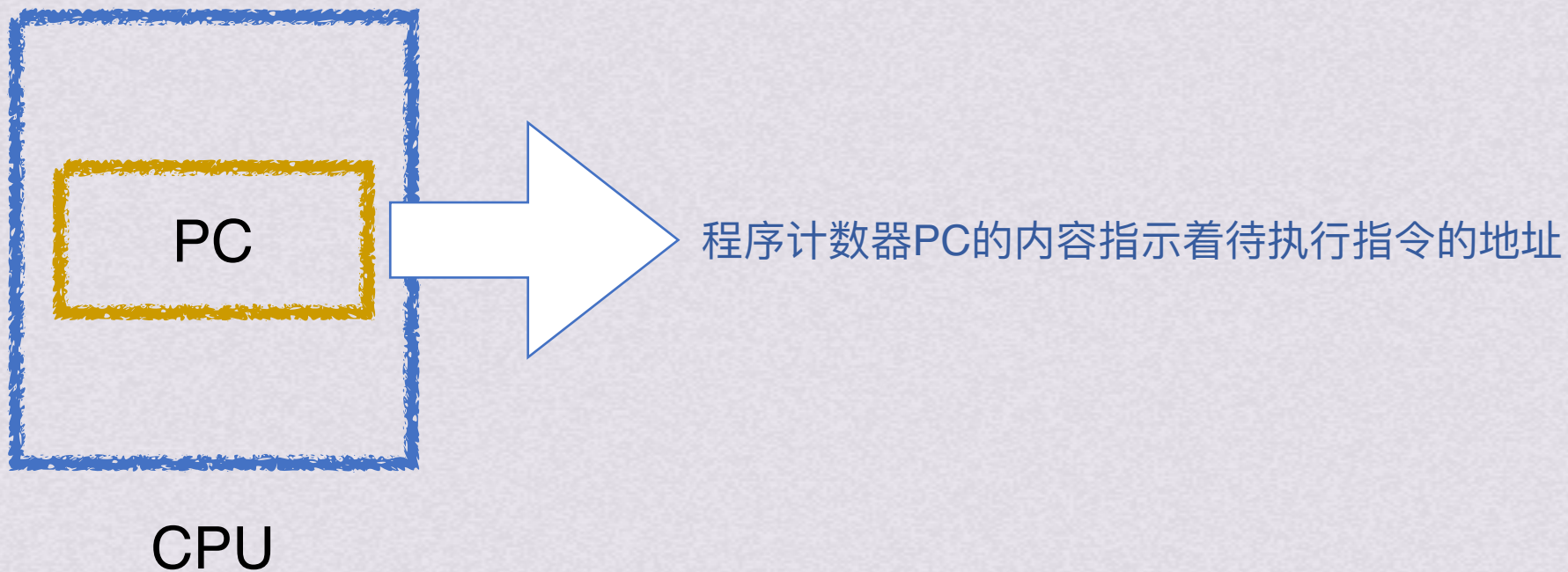
1.指令的寻址方式



寻址方式

1. 指令的寻址方式

寻找下一条将要执行的指令地址称为指令寻址





寻址方式

1. 指令的寻址方式

寻找下一条将要执行的指令地址称为指令寻址

从指令1开始顺序执行指令



PC

指令1	8000 0000H
指令2	8000 0001H
指令3	8000 0002H
指令4	8000 0003H

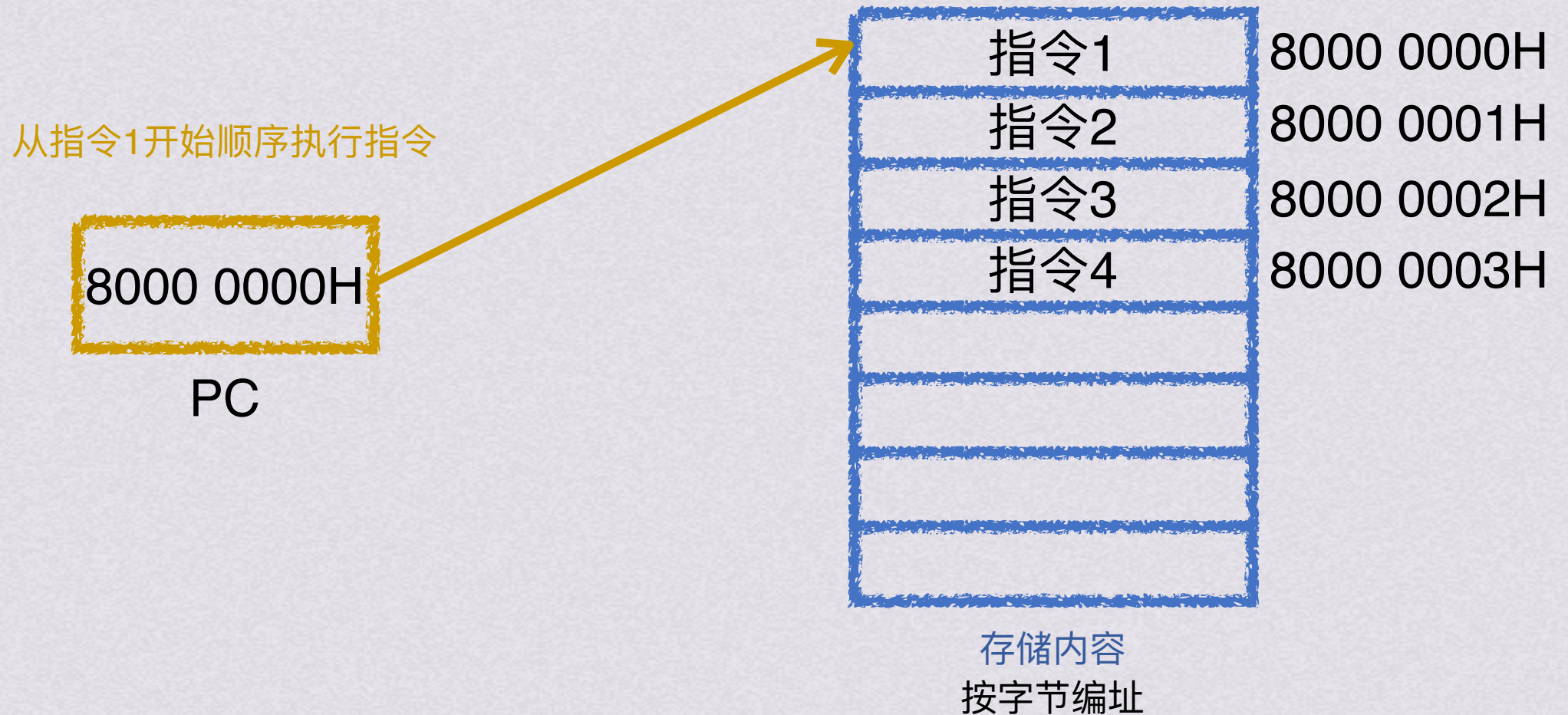
存储内容
按字节编址



寻址方式

1. 指令的寻址方式

寻找下一条将要执行的指令地址称为指令寻址





寻址方式

1. 指令的寻址方式

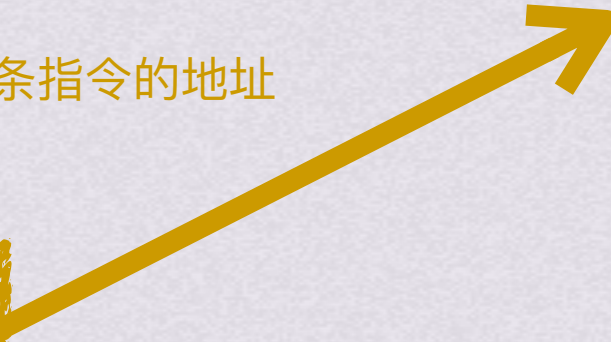
寻找下一条将要执行的指令地址称为指令寻址

指令1取出后,PC自增,计算下一条指令的地址

$PC = PC + 1$

8000 0000H

PC



指令1	8000 0000H
指令2	8000 0001H
指令3	8000 0002H
指令4	8000 0003H

存储内容
按字节编址



寻址方式

1. 指令的寻址方式

寻找下一条将要执行的指令地址称为指令寻址

指令1取出后,PC自增,计算下一条指令的地址

$PC = PC + 1$

8000 0001H

PC



指令1	8000 0000H
指令2	8000 0001H
指令3	8000 0002H
指令4	8000 0003H

存储内容
按字节编址



寻址方式

1. 指令的寻址方式

寻找下一条将要执行的指令地址称为指令寻址

一旦取出指令,PC就会自动计算下一条指令的地址

8000 0001H

PC



指令1	8000 0000H
指令2	8000 0001H
指令3	8000 0002H
指令4	8000 0003H

存储内容
按字节编址



寻址方式

1. 指令的寻址方式

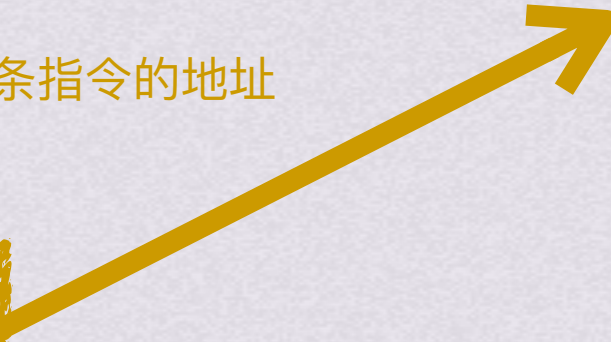
寻找下一条将要执行的指令地址称为指令寻址

指令1取出后,PC自增,计算下一条指令的地址

$$PC = PC + 4$$

8000 0000H

PC



指令1	8000 0000H
指令2	8000 0004H
指令3	8000 0008H
指令4	8000 000CH

存储内容

按字节编址 指令字长=4B



寻址方式

1. 指令的寻址方式

寻找下一条将要执行的指令地址称为指令寻址

指令1取出后,PC自增,计算下一条指令的地址

$$PC = PC + 4$$

8000 0004H

PC



指令1	8000 0000H
指令2	8000 0004H
指令3	8000 0008H
指令4	8000 000CH

存储内容

按字节编址 指令字长=4B



寻址方式

1.指令的寻址方式

寻找下一条将要执行的指令地址称为指令寻址

从指令1开始执行指令,指令1是跳转指令

8000 0000H

PC

指令1(jmp 2)	8000 0000H
指令2	8000 0001H
指令3	8000 0002H
指令4	8000 0003H

存储内容
按字节编址



2.数据的寻址方式

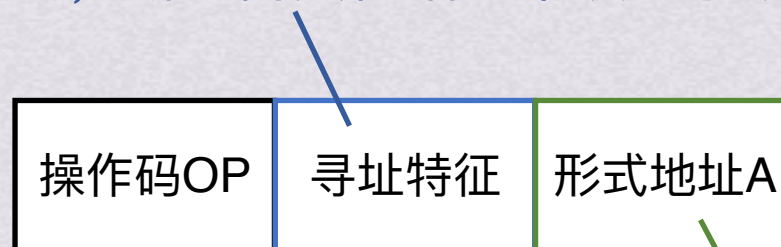


寻址方式

2.数据的寻址方式

寻找指令中的操作数的地址称为数据寻址

因为数据寻址方式太多，所以需要在指令中设一字段指出属于哪一种方式



一种一地址指令的格式

因为通常指令的地址字段不是真实地址
所以叫形式地址
真实地址叫有效地址，记EA



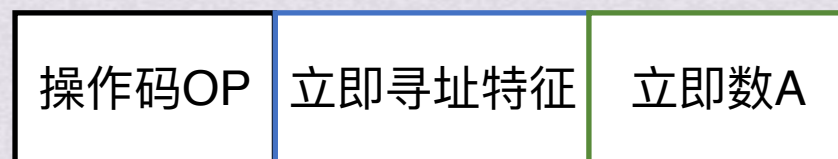
寻址方式

2.数据的寻址方式

2-1立即寻址

操作数直接写在指令字内，形式地址A不是操作数的地址，而是操作数本身，又称为立即数

数据以补码形式存放



优点：只要取出指令，可立即获得操作数，这样就不用访问存储器

缺点：A的位数限制了能表示的立即数的范围



寻址方式

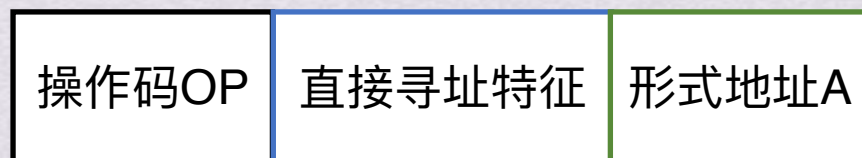
2-1立即寻址

2.数据的寻址方式

2-2直接寻址

指令字中的形式地址A就是操作数的真实地址EA

$$EA=A$$





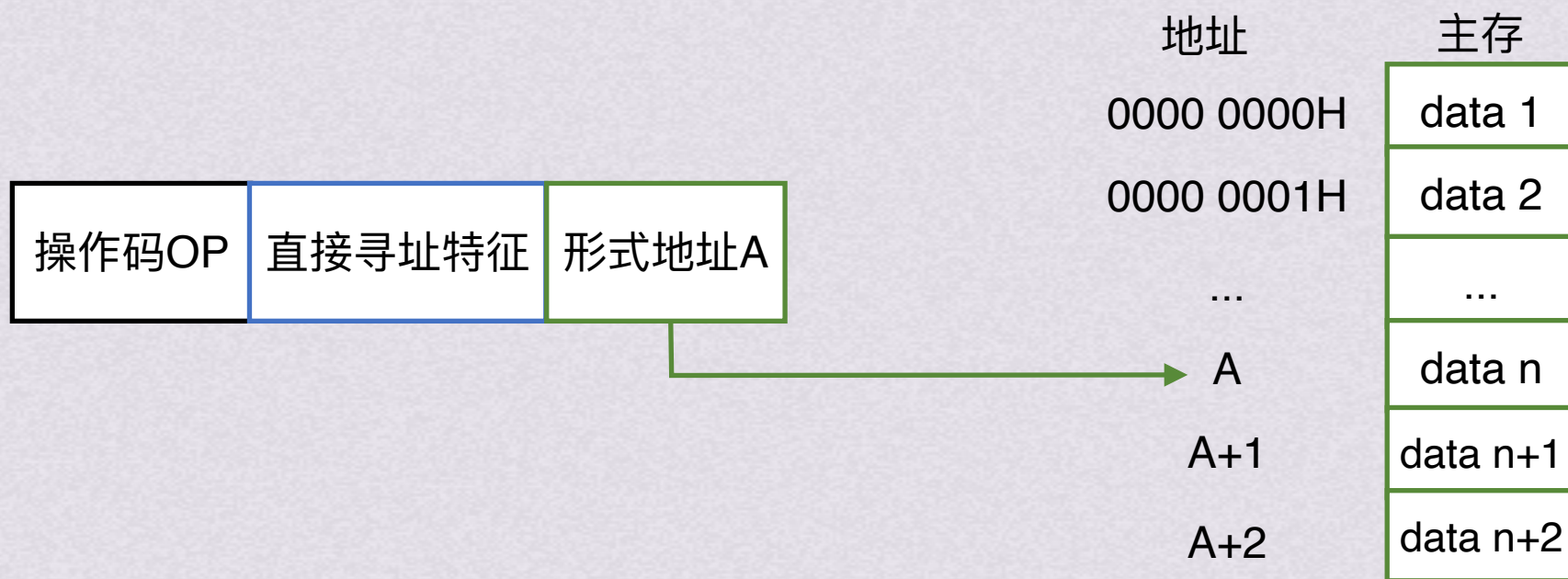
寻址方式

2-1立即寻址
2.数据的寻址方式

2-2直接寻址

指令字中的形式地址A就是操作数的真实地址EA

$$EA=A$$



优点：寻找操作数比较简单，也不需要专门计算操作数的地址，指令执行阶段对主存只访问一次

缺点：A的位数限制了操作数的寻址范围，且必须修改A的值，才能修改操作数的地址



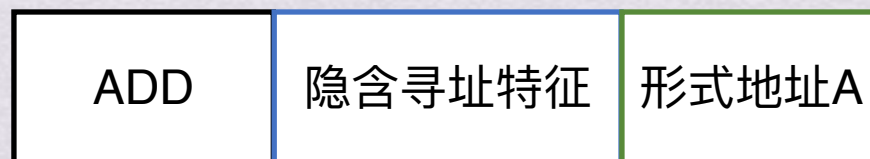
寻址方式

2-1立即寻址 2-2直接寻址

2.数据的寻址方式

2-3隐含寻址

指令字中不明显地给出操作数的地址，其操作数的地址隐含在操作码或某个寄存器中





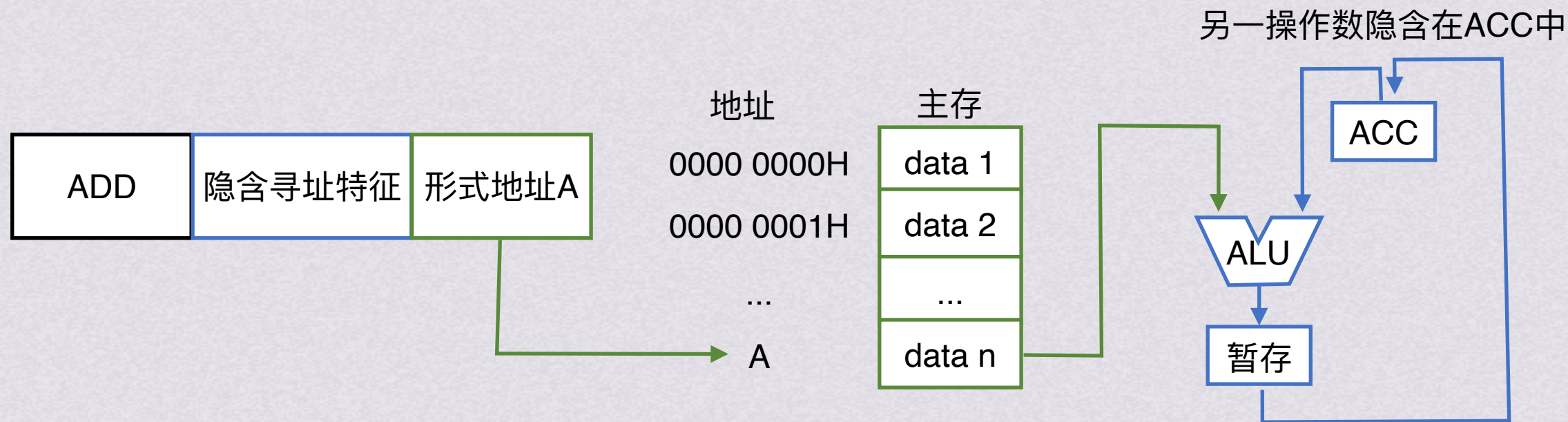
寻址方式

2-1立即寻址 2-2直接寻址
2.数据的寻址方式

2-3隐含寻址

指令字中不明显地给出操作数的地址，其操作数的地址隐含在操作码或某个寄存器中

如：一地址格式加法指令只给出一个操作数的地址，另一个操作数隐含在累加器ACC中
这样累加器ACC成了另一个数的地址



优点：指令字中少个地址，能缩短指令字长

缺点：需增加存储操作数或隐含地址的硬件



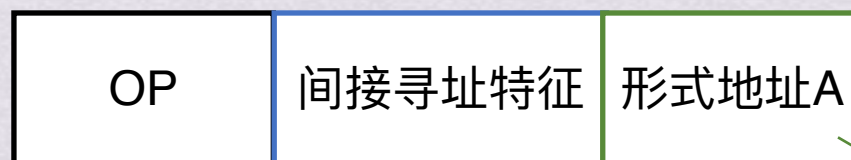
寻址方式

2-1立即寻址 2-2直接寻址 2-3隐含寻址

2.数据的寻址方式

2-4间接寻址

指令字中形式地址不直接指出操作数地址，而是给出操作数有效地址所在地址



给的是地址的地址

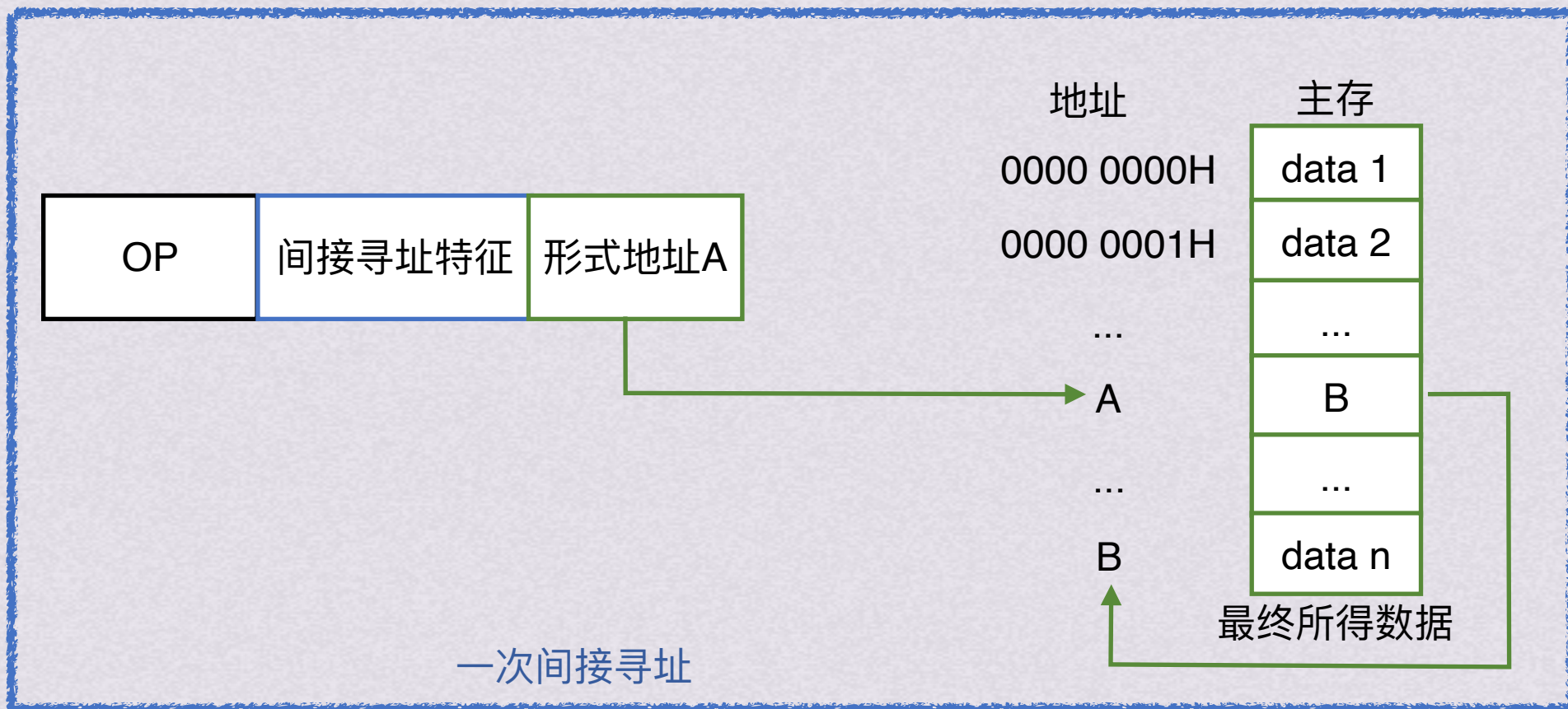


寻址方式

2-1立即寻址 2-2直接寻址 2-3隐含寻址
2.数据的寻址方式

2-4间接寻址

指令字中形式地址不直接指出操作数地址，而是给出操作数有效地址所在地址



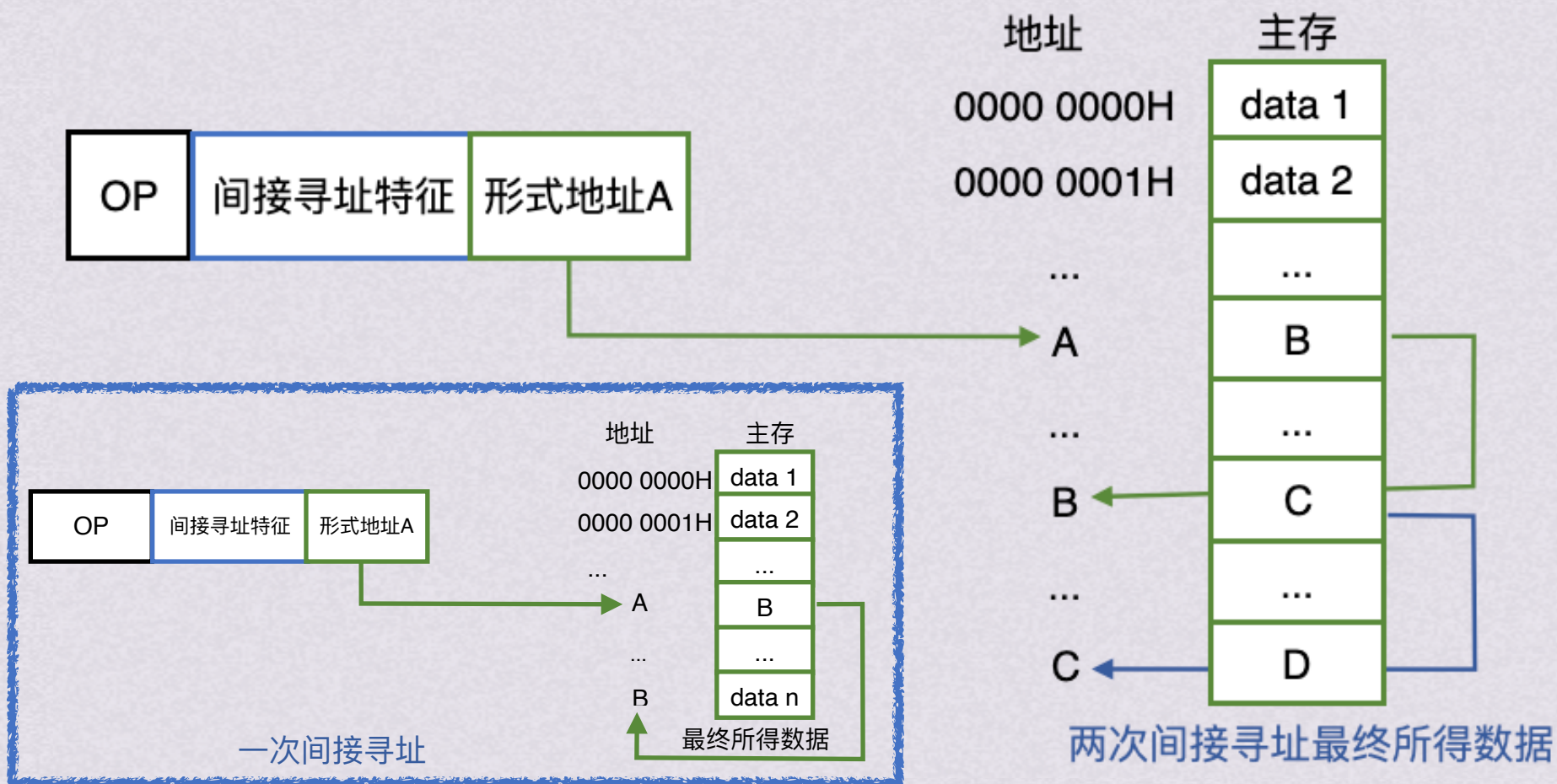


寻址方式

2-1立即寻址 2-2直接寻址 2-3隐含寻址
2.数据的寻址方式

2-4间接寻址

指令字中形式地址不直接指出操作数地址，而是给出操作数有效地址所在地址





寻址方式

2-1立即寻址 2-2直接寻址 2-3隐含寻址

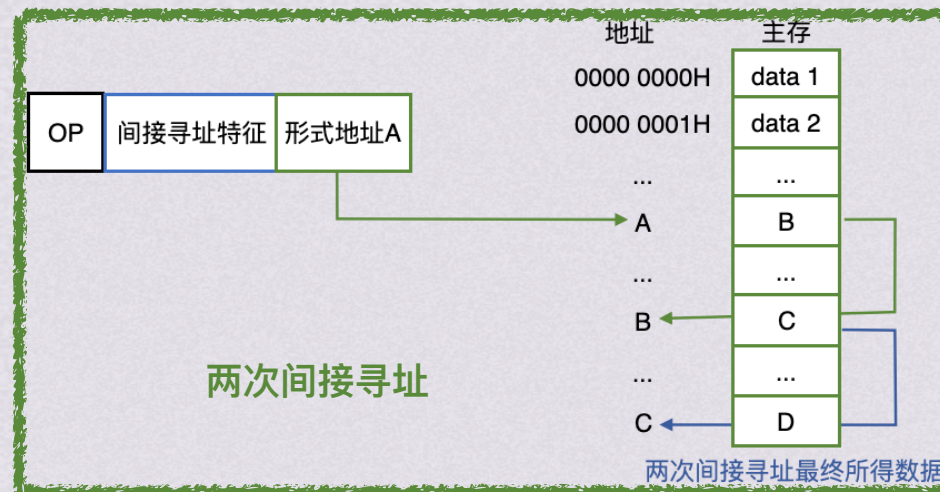
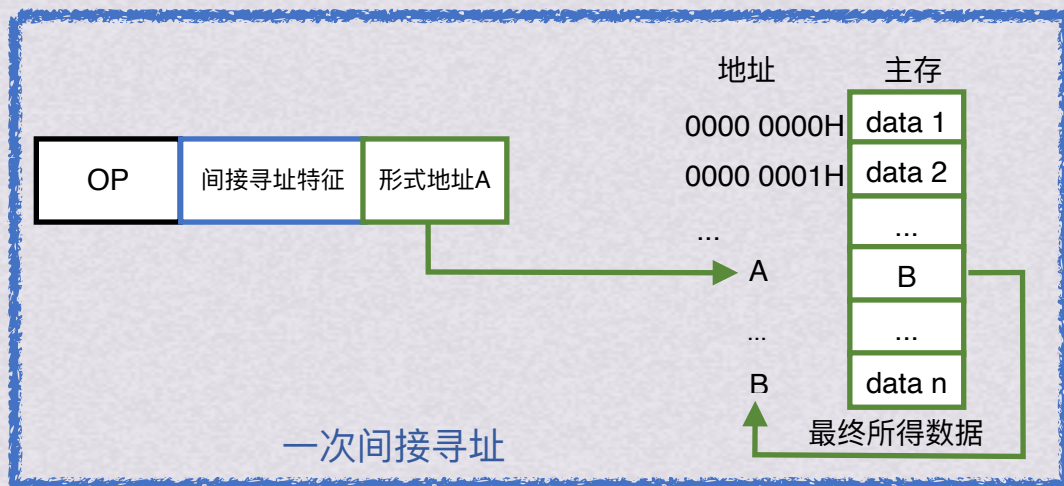
2.数据的寻址方式

2-4间接寻址

指令字中形式地址不直接指出操作数地址，而是给出操作数有效地址所在地址

优点：可扩大寻址范围（有效地址EA位数大于形式地址A位数）

缺点：指令在执行阶段需要多次访存





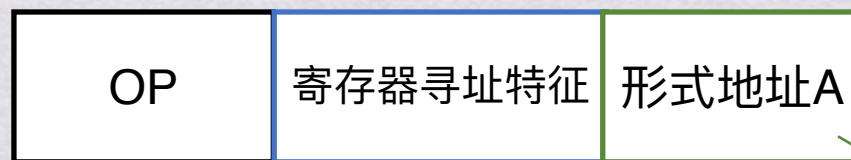
寻址方式

2-1立即寻址 2-2直接寻址 2-3隐含寻址 2-4间接寻址

2.数据的寻址方式

2-5寄存器寻址

地址码字段给出寄存器编号，即 $EA=R$



给的是寄存器编号



寻址方式

2-1立即寻址 2-2直接寻址 2-3隐含寻址 2-4间接寻址

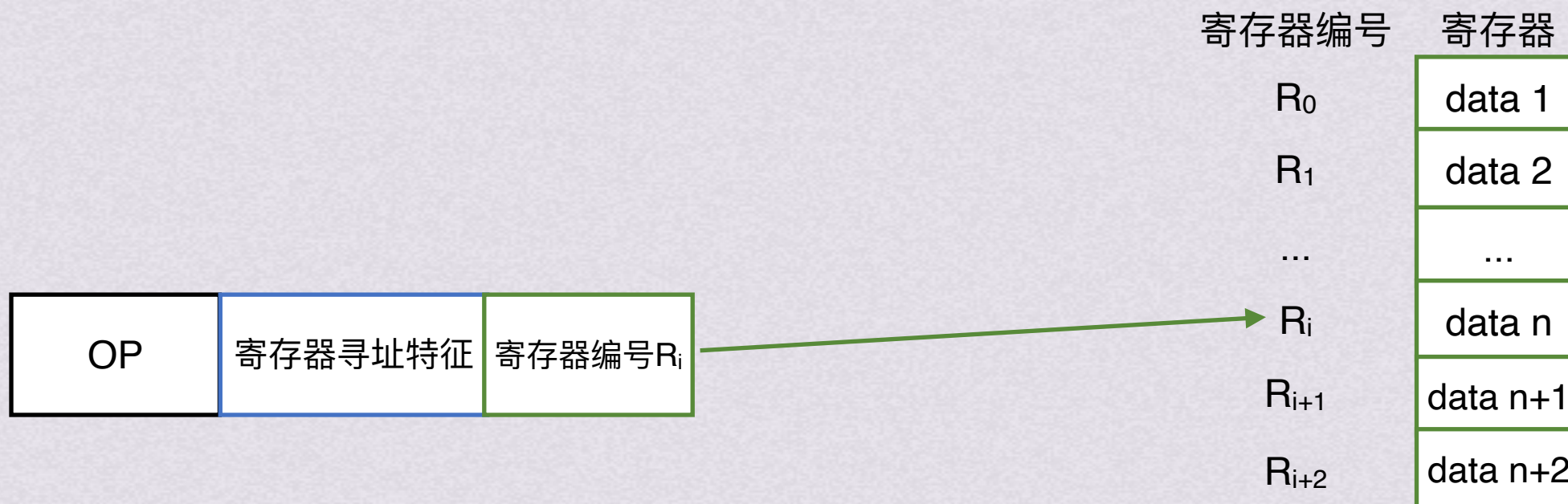
2.数据的寻址方式

2-5寄存器寻址

地址码字段给出寄存器编号，即 $EA=R$

优点：指令执行阶段无需访存，减少了执行时间

缺点：没啥缺点，就是贵点，使用相当广泛





寻址方式

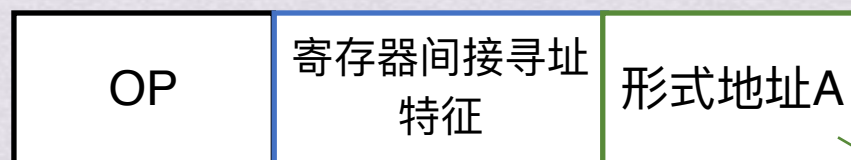
2-1立即寻址 2-2直接寻址 2-3隐含寻址 2-4间接寻址 2-5寄存器寻址

2.数据的寻址方式

2-6寄存器间接寻址

综合了寄存器寻址和间接寻址的特点，指令字中给出寄存器编号，寄存器中记录的是操作数所在主存单元的地址
即有效地址 $EA=(R_i)$

(R_i) 是 取出寄存器 R_i 中的值 的意思



给的是寄存器编号

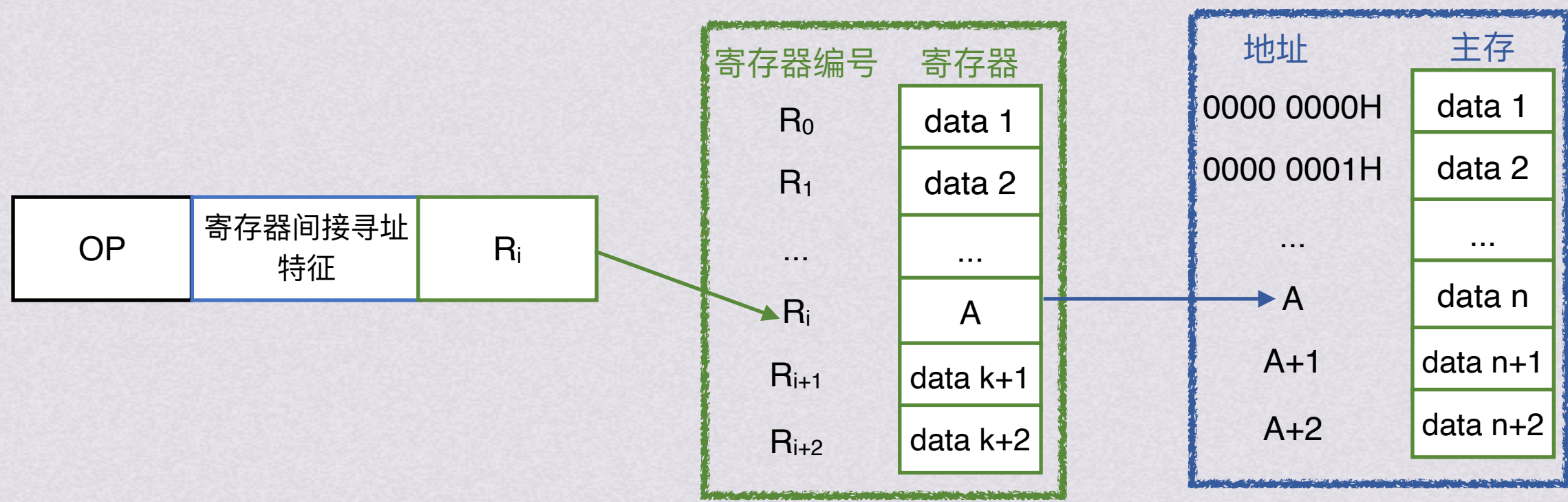
寻址方式

2-1立即寻址 2-2直接寻址 2-3隐含寻址 2-4间接寻址 2-5寄存器寻址

2.数据的寻址方式

2-6寄存器间接寻址

综合了寄存器寻址和间接寻址的特点，指令字中给出寄存器编号，寄存器中记录的是操作数所在主存单元的地址
即有效地址 $EA=(R_i)$
(R_i)是 取出寄存器 R_i 中的值 的意思



指令执行阶段需要访存一次
比间接寻址少访存一次



寻址方式

2-1立即寻址 2-2直接寻址 2-3隐含寻址 2-4间接寻址 2-5寄存器寻址 2-6寄存器间接寻址

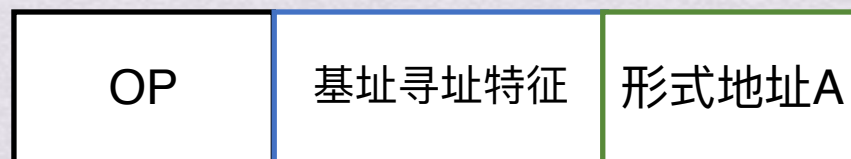
2.数据的寻址方式

2-7基址寻址

基址寻址需要有基址寄存器BR，操作数的有效地址EA=指令字中的形式地址与基址寄存器中内容相加

$$EA = A + (BR)$$

(BR)是 取出寄存器BR中的值 的意思





寻址方式

2-1立即寻址 2-2直接寻址 2-3隐含寻址 2-4间接寻址 2-5寄存器寻址 2-6寄存器间接寻址

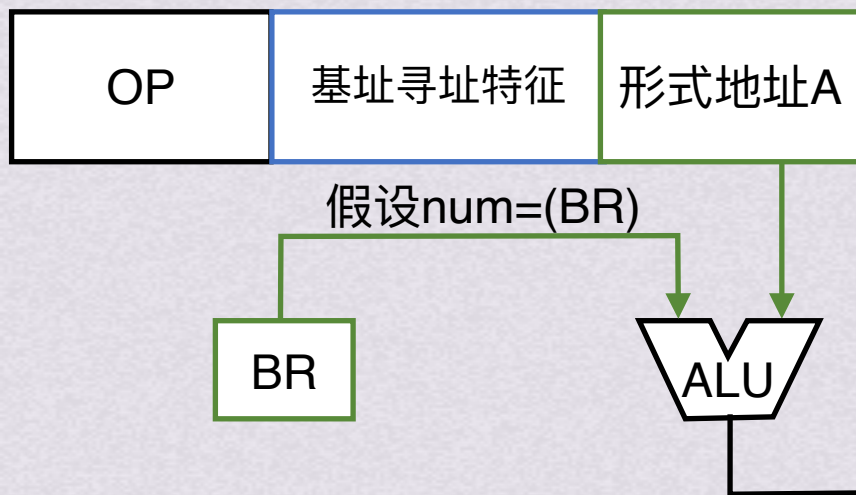
2.数据的寻址方式

2-7基址寻址

基址寻址需要有基址寄存器BR，操作数的有效地址EA=指令字中的形式地址与基址寄存器中内容相加

$$EA = A + (BR)$$

(BR)是 取出寄存器BR中的值 的意思



地址	主存
0000 0000H	data 1
0000 0001H	data 2
...	...
A+num	data n
A+num+1	data n+1
A+num+2	data n+2

基址寻址扩大寻址范围，有利于多道程序设计，可用于编制浮动程序

有隐式和显式两种

隐式：计算机内专门设有一个基址寄存器BR，使用时用户不必明显指出该寄存器

显式：一组通用寄存器里，用户明确指出哪个寄存器用做基址寄存器，存放基地址



寻址方式

2-1立即寻址 2-2直接寻址 2-3隐含寻址 2-4间接寻址 2-5寄存器寻址 2-6寄存器间接寻址

2.数据的寻址方式

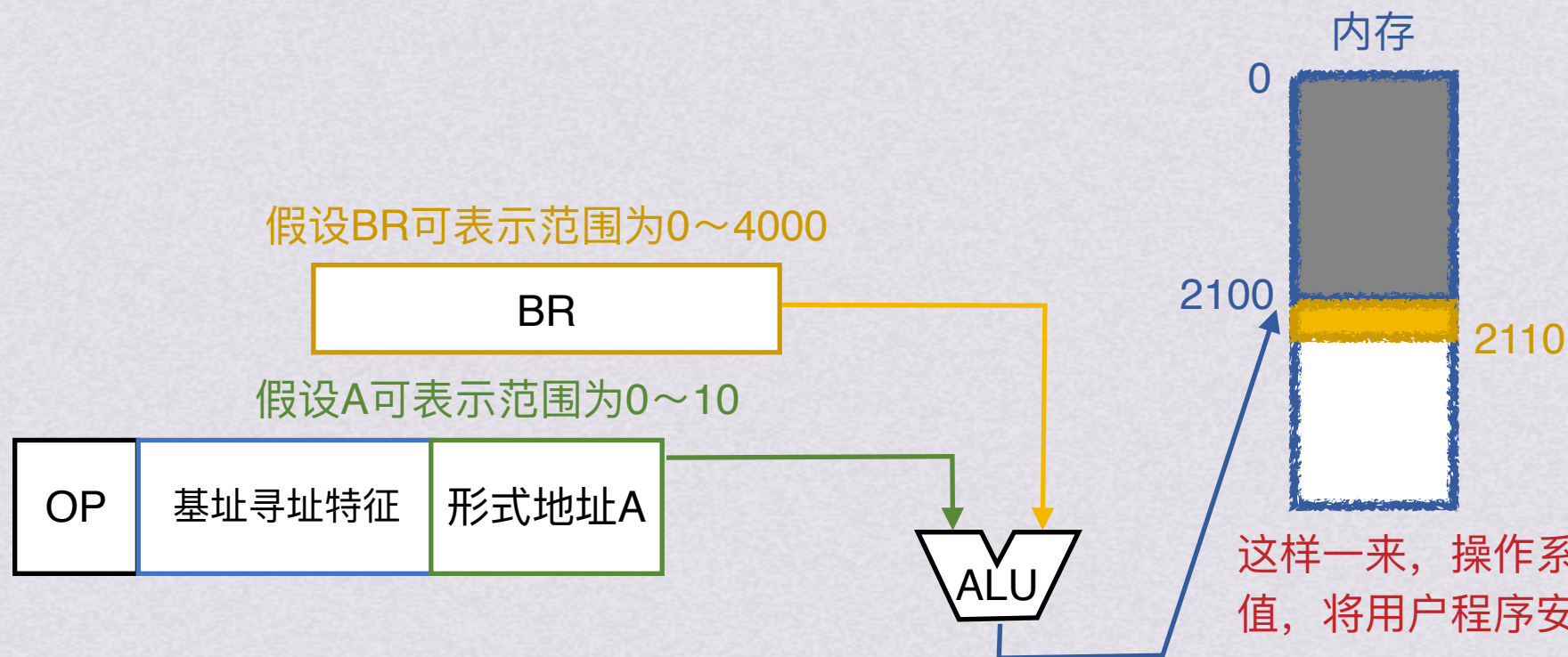
2-7基址寻址

基址寻址需要有基址寄存器BR，操作数的有效地址EA=指令字中的形式地址与基址寄存器中内容相加

$$EA = A + (BR)$$

(BR)是 取出寄存器BR中的值 的意思

基址寄存器是面向操作系统的，只能由操作系统或管理程序根据主存空间情况来确定



这样一来，操作系统就可以通过给基址寄存器赋值，将用户程序安置在主存的某一空间区域



寻址方式

2-1立即寻址 2-2直接寻址 2-3隐含寻址 2-4间接寻址 2-5寄存器寻址 2-6寄存器间接寻址 2-7基址寻址

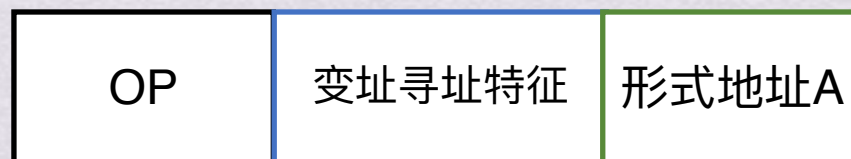
2.数据的寻址方式

2-8变址寻址

与基址寻址极为相似，有效地址EA=指令字中形式地址A与变址寄存器IX内容相加之和

$$EA = A + (IX)$$

(IX)是 取出寄存器IX中的值 的意思





寻址方式

2-1立即寻址 2-2直接寻址 2-3隐含寻址 2-4间接寻址 2-5寄存器寻址 2-6寄存器间接寻址 2-7基址寻址

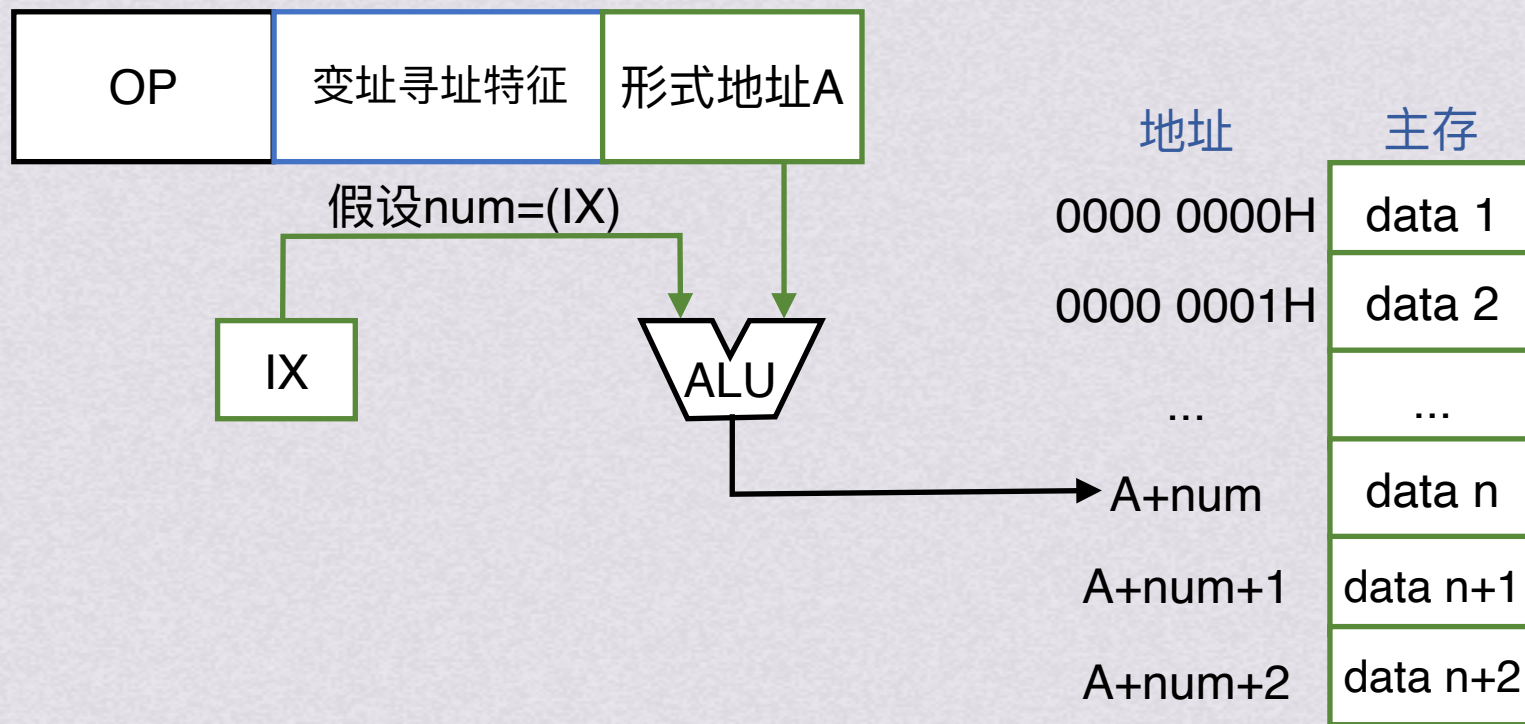
2.数据的寻址方式

2-8变址寻址

与基址寻址极为相似，有效地址EA=指令字中形式地址A与变址寄存器IX内容相加之和

$$EA = A + (IX)$$

(IX)是 取出寄存器IX中的值 的意思



寻址方式

2-1立即寻址 2-2直接寻址 2-3隐含寻址 2-4间接寻址 2-5寄存器寻址 2-6寄存器间接寻址 2-7基址寻址

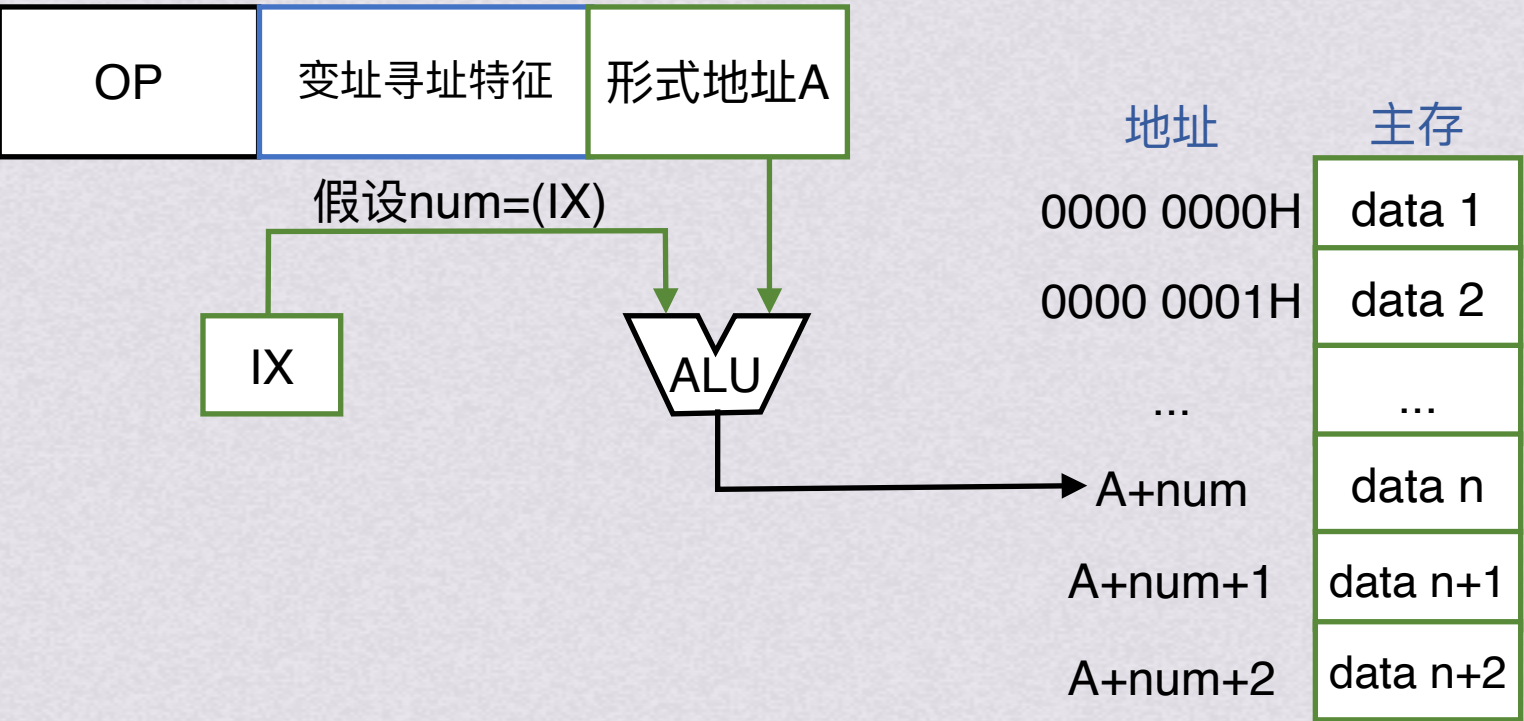
2.数据的寻址方式

2-8变址寻址

与基址寻址极为相似，有效地址EA=指令字中形式地址A与变址寄存器IX内容相加之和

$EA = A + (IX)$

(IX)是 取出寄存器IX中的值 的意思



基址寄存器内容由操作系统或管理程序确定，程序执行过程中其值不可变，指令字中A可变

变址寄存器的内容由用户设定，程序执行过程中其值可变，而指令字中A不可变

变址寻址主要用于处理数组问题，可设定A为数组首地址，不断改变变址寄存器IX的内容，可很容易形成数组中任一数据地址，适合编制循环程序



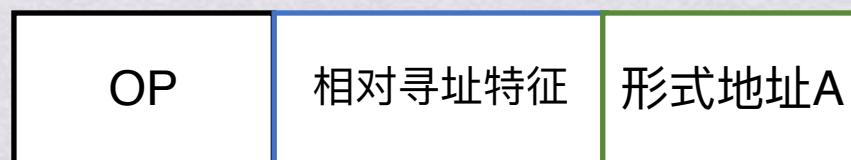
寻址方式

2-1立即寻址 2-2直接寻址 2-3隐含寻址 2-4间接寻址 2-5寄存器寻址 2-6寄存器间接寻址 2-7基址寻址 2-8变址寻址

2.数据的寻址方式

2-9相对寻址

相对寻址的有效地址是将程序计数器PC的内容（即当前指令下一条指令的地址）与指令字中的形式地址A相加而成

$$EA = (PC) + A$$


寻址方式

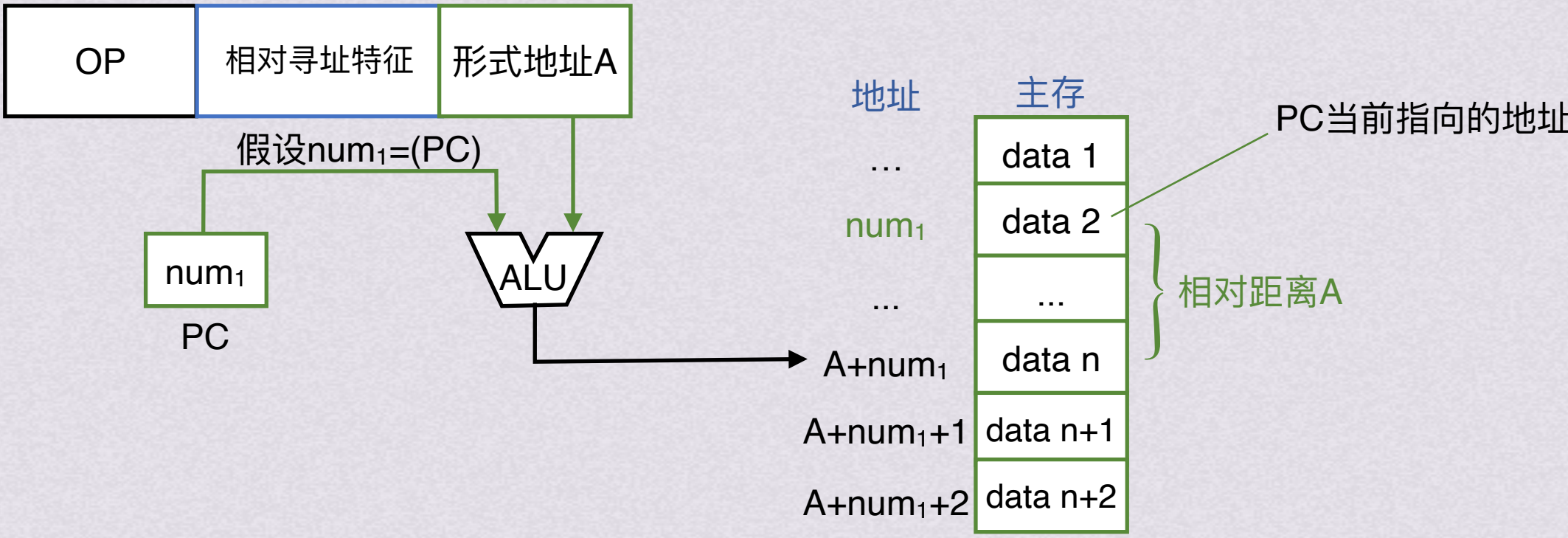
2-1立即寻址 2-2直接寻址 2-3隐含寻址 2-4间接寻址 2-5寄存器寻址 2-6寄存器间接寻址 2-7基址寻址 2-8变址寻址

2.数据的寻址方式

2-9相对寻址

相对寻址的有效地址是将程序计数器PC的内容（即当前指令下一条指令的地址）与指令字中的形式地址A相加而成

$EA = (PC) + A$





寻址方式

2-1立即寻址 2-2直接寻址 2-3隐含寻址 2-4间接寻址 2-5寄存器寻址 2-6寄存器间接寻址 2-7基址寻址 2-8变址寻址

2.数据的寻址方式

2-9相对寻址

相对寻址的有效地址是将程序计数器PC的内容（即当前指令下一条指令的地址）与指令字中的形式地址A相加而成

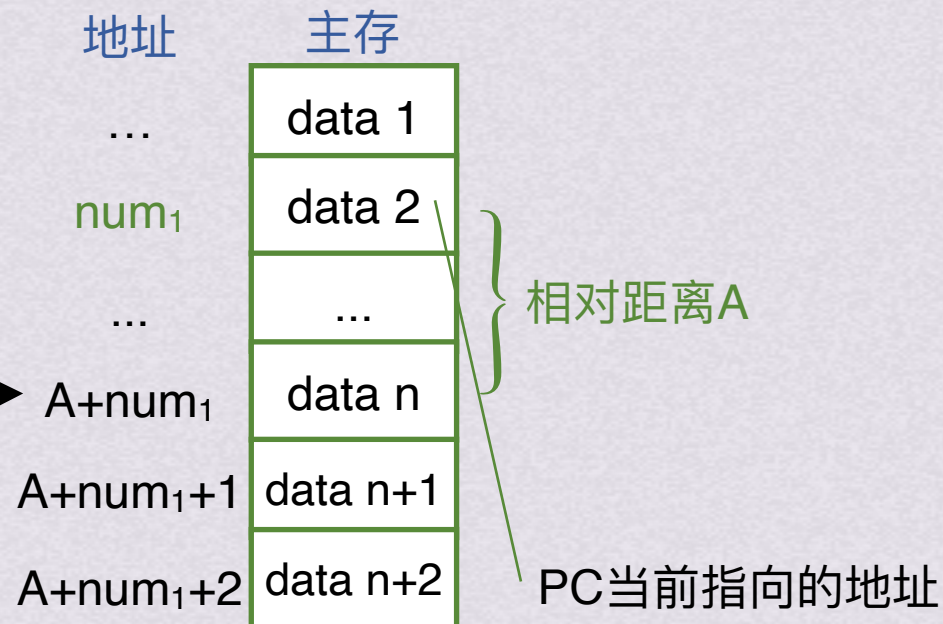
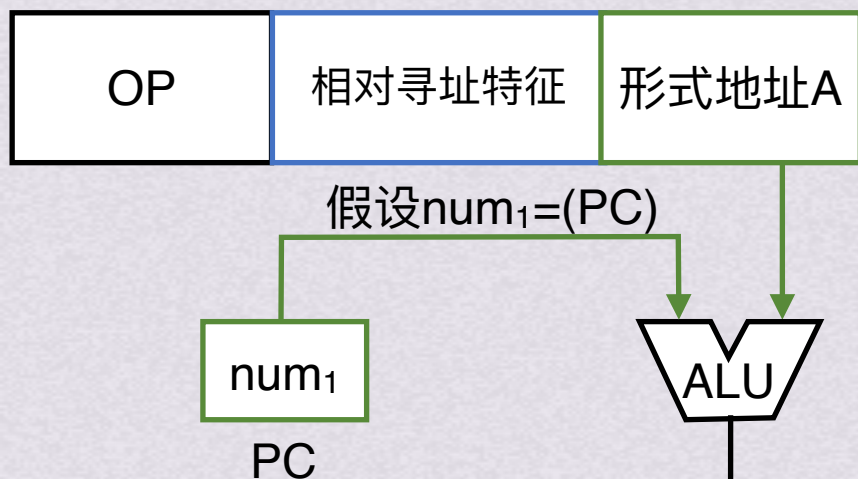
$$EA = (PC) + A$$

相对寻址通常被用于转移类指令，转移后的目标地址与当前指令有一段距离，称相对位移量

相对位移量由形式地址A给出，所以A又成位移量

A可正可负，常用补码表示

假设A为8位补码，则指令寻址地址在 $(PC)+127 \sim (PC)-128$ 之间





寻址方式

2.数据的寻址方式

数据的寻址方式

2-1立即寻址 直接放数

2-2直接寻址 直接放地址

2-3隐含寻址 藏一个地址（隐含的给出）

2-4间接寻址 放地址的地址

2-5寄存器寻址 指出寄存器，数放寄存器

2-6寄存器间接寻址 指出寄存器，地址放寄存器

2-7基址寻址 基址寄存器，面向OS

2-8变址寻址 变址寄存器，面向用户

2-9相对寻址 PC的值+给出的A=目的地址